

Simulation-two-way

- 1. Sparsity on v (1-SE)
- 2. Smoothness on v
- 3. Smoothness and Sparsity on v
- 4.1. Sparsity on u (1-SE)
- 4.2. Smoothness on u
- 5. Two-way conditional sparsity
- 6. Two-way conditional smoothness
- 7. Conditional two-way sparsity two-way smoothness
- 8. Multivariate hybrid data case

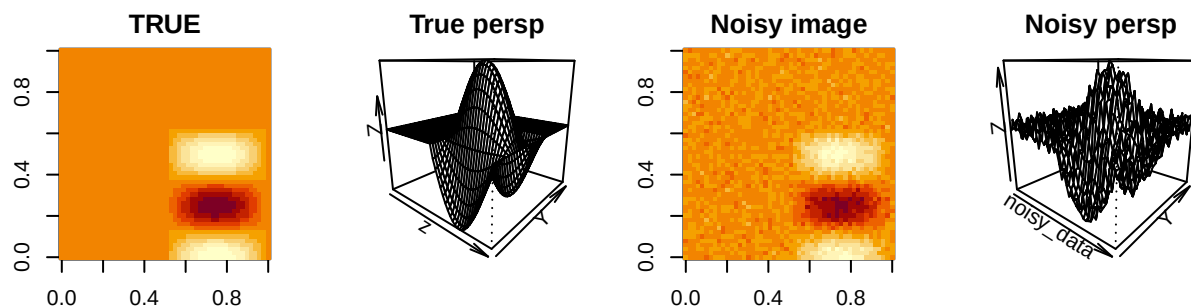
4.1. Sparsity on u (1-SE)

To test Sparsity and Smoothness on u , we need a set of data that has functional behavior on both sides to be able to incorporating regularization on both the left and right singular vectors in the Singular Value Decomposition (SVD) of the data matrix. As a result, we can work with the following data:

```
x <- seq(0,2*pi, len = 50)
y1 <- sin(x); y1[1:26] <- 0
y2 <- cos(2*x); y2[32:50] <- 0
z<- outer(y1,y2)

par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(z, main = "TRUE")
persp(z, theta = 40, main = "True persp")

set.seed(123)
noisy_data <- z + rnorm(length(z), sd = 0.1)
image(noisy_data, main = "Noisy image")
persp(noisy_data, theta = 40, main = "Noisy persp")
```



Let's start with the sparsity on v for this image that we've already tested it!

```
fd_object <- fdClass(data = as.matrix(noisy_data),
  argval = NULL,
  Smoothing_parameter = NULL,
  Sparsity_parameter = round(seq(0,49, length.out = 30)))

hd_list <- list(fd_object) # List of fd and rd objects
object_list <- hdClass(hdlist = hd_list,
  argval = NULL, # Grid points for rows
  Smoothing_parameter = NULL, # Custom smoothing parameter for rows
  Sparsity_parameter = round(seq(0,49, length.out = 40))) # Custom sparsity param

# Combine matrices side by side
hd <- object_list
n <- nrow(hd); K_fold <- 5; sparse_tuning_type <- "soft"
n_var <- attr(object_list, "n_var") # Number of variables (# of matrices in object_list)
ncol <- data.frame(attr(object_list, "ncol")) # Number of columns of each matrix

##### Smoothing Parameter #####
# Generate all combinations alphas (one row per combination)
smooth_tuning_col <- expand.grid(attr(object_list, "Smoothing_parameter_col")) # A matrix
smooth_tuning_row <- attr(object_list, "Smoothing_parameter") # A vector

##### level of sparsity (for both functional and non-functional data) #####
sparsity_row_list <- attr(object_list, "Sparsity_parameter")
```

```

sparsity_col_list <- attr(object_list, "Sparsity_parameter_col")

X_temp <- hd
# No smoothness list:
S_alpha_v <- list()
for (i in 1:n_var) {
  S_alpha_v[[i]] <- diag(ncol[,i])
}

cv_sparse_col_SE <- function(data,
                             n_var,
                             ncol,
                             S_alpha_v,
                             S_alpha_u,
                             K_fold,
                             sparse_tuning_result_u,
                             sparse_tuning_result_v,
                             sparse_tuning_type) {

  set.seed(123)
  shuffled_rows <- sample(nrow(data)) # Grouping rows of data matrix
  group_size <- ifelse(round(length(shuffled_rows) / K_fold, digits = 0) == 0,
                       1,
                       round(length(shuffled_rows) / K_fold, digits = 0))

  data_tilde <- data
  fold_errors <- numeric(K_fold) # store errors

  for (k in 1:K_fold) {
    rows_to_remove <- shuffled_rows[((k - 1) * group_size + 1):min(k * group_size, length(shuffled_rows))]
    data_train <- data.frame(data_tilde[-rows_to_remove,])
    data_test <- data.frame(matrix(data_tilde[rows_to_remove,], nrow = length(rows_to_remove)))

    # Power Algorithm
    power_train <- power_algo(data = data_train,
                              n_var = n_var,
                              ncol = ncol,
                              sparse_tuning_result_u = sparse_tuning_result_u,
                              sparse_tuning_result_v = sparse_tuning_result_v,
                              S_alpha_v = S_alpha_v,
                              S_alpha_u = S_alpha_u[-rows_to_remove, -rows_to_remove],
                              sparse_tuning_type = sparse_tuning_type)

    v_train <- power_train[[1]]
    u_test <- as.matrix(data_test) %*% as.matrix(v_train)

    # Fold-wise error
    fold_errors[k] <- sum((data_test - u_test %*% t(v_train))^2) / nrow(data_test)
  }
  CV_mean <- mean(fold_errors)
  CV_se <- sd(fold_errors) / sqrt(K_fold)

  return(list(CV_error = CV_mean, SE = CV_se, fold_errors = fold_errors))

```

```

}

# Tuning across sparsity levels using 1-SE rule
CV_score_sparse_u <- CV_score_sparse_v <- GCV_score_smooth_u <- GCV_score_smooth_v <- Inf
CV_scores_result_u <- CV_scores_result_v <- list()
CV_results <- list()

gamma_v <- rep(0, n_var)
for (i in 1:n_var) {
  gamma_Xi <- sparsity_col_list[[i]]
  cv_means <- cv_ses <- fold_errors_vec <- numeric(length(gamma_Xi))

  for (j in seq_along(gamma_Xi)) {
    gamma_v[i] <- gamma_Xi[j]

    cv_result <- cv_sparse_col_SE(data = X_temp,
                                  n_var = n_var,
                                  ncol = ncol,
                                  S_alpha_v = S_alpha_v,
                                  S_alpha_u = diag(nrow(X_temp)),
                                  K_fold = K_fold,
                                  sparse_tuning_result_u = 0,
                                  sparse_tuning_result_v = gamma_v,
                                  sparse_tuning_type = sparse_tuning_type)

    cv_means[j] <- cv_result$CV_error
    cv_ses[j] <- cv_result$SE
    #fold_errors_vec <- cv_result$fold_errors
  }

  CV_results[[i]] <- data.frame(gamma_Xi, cv_means, cv_ses)

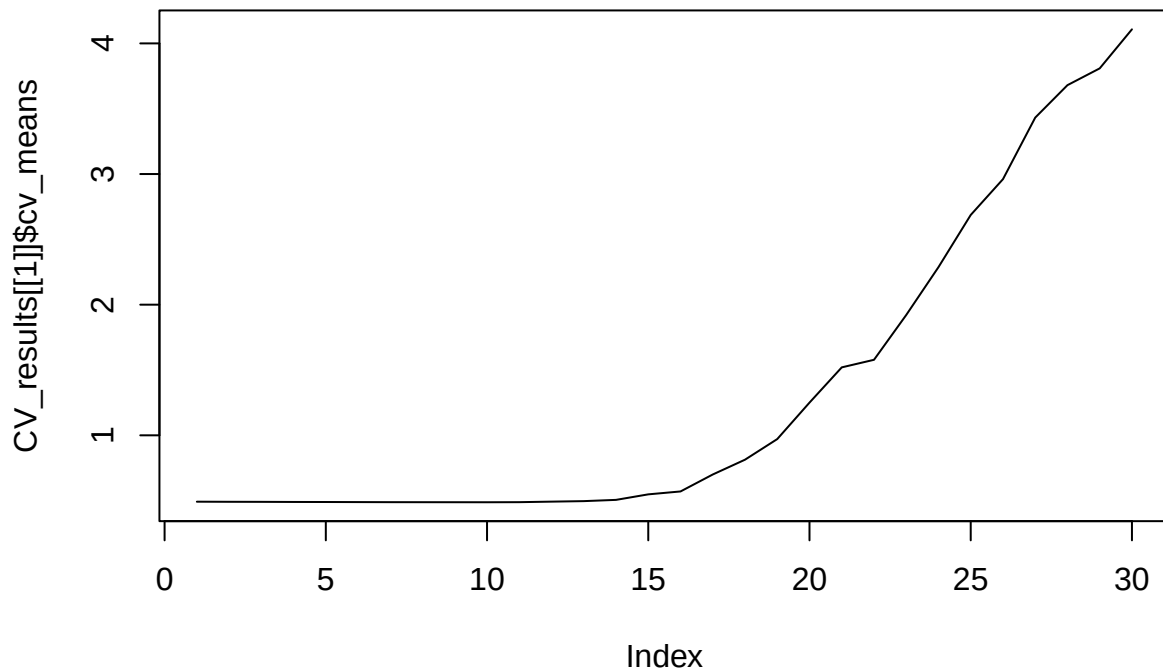
  # 1-SE rule
  j_min <- which.min(cv_means)
  threshold <- cv_means[j_min] + cv_ses[j_min]
  j_1se <- max(which(cv_means <= threshold)) # Most sparse within 1-SE

  # Store results
  CV_scores_result_v[[i]] <- list(errors = cv_means, SEs = cv_ses)
  sparse_tuning_selection_v <- gamma_Xi[j_1se]
  gamma_v[i] <- sparse_tuning_selection_v
}
gamma_v

## [1] 19

plot(CV_results[[1]]$cv_means, type = 'l')

```



```

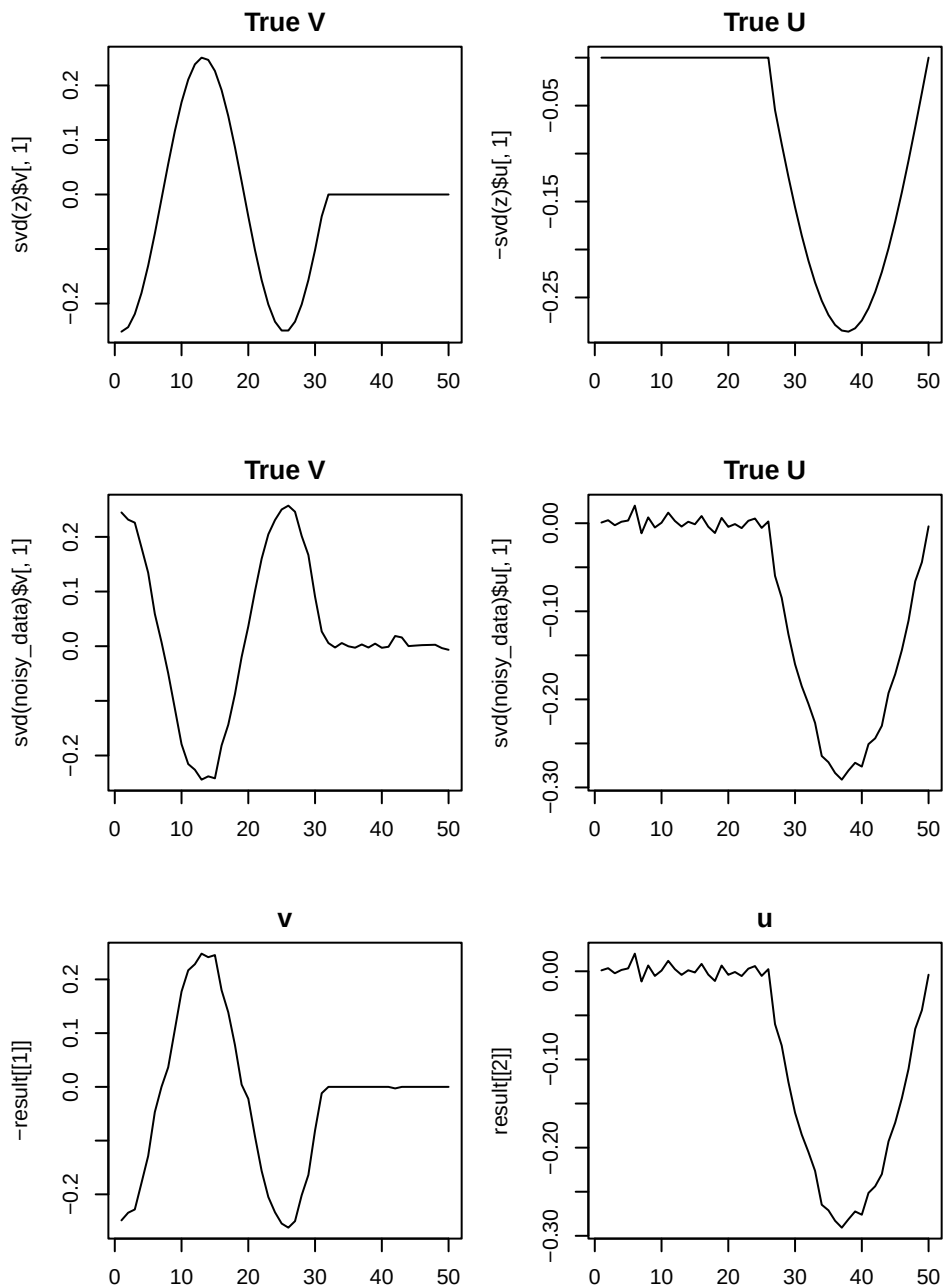
par(mfrow = c(3,2), mar = c(4, 4, 2, 1))
matplot(svd(z)$v[,1], type = 'l', main = "True V")
matplot(-svd(z)$u[,1], type = 'l', main = "True U")

matplot(svd(noisy_data)$v[,1], type = 'l', main = "True V")
matplot(svd(noisy_data)$u[,1], type = 'l', main = "True U")

result <- power_algo(data = X_temp,
                     n_var = n_var,
                     ncol = ncol,
                     sparse_tuning_result_u = 0, # A fixed number
                     sparse_tuning_result_v = gamma_v, # A vector (length = p)
                     S_alpha_v = S_alpha_v, # A list (length = p)
                     S_alpha_u = diag(nrow(X_temp)), # A matrix
                     sparse_tuning_type = sparse_tuning_type)

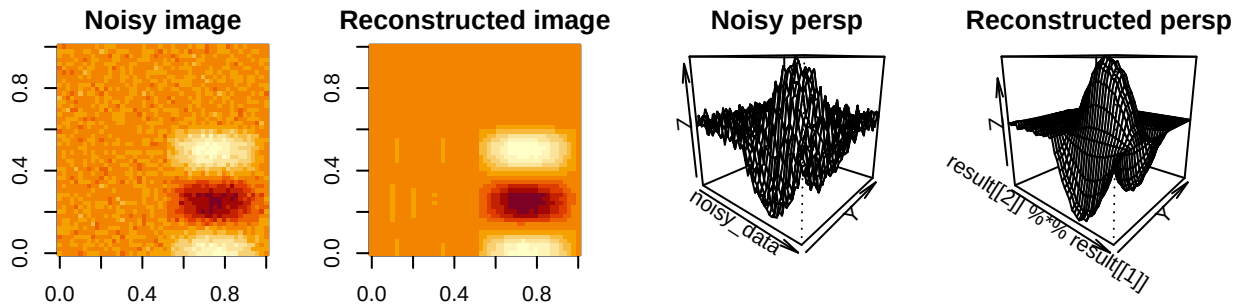
matplot(-result[[1]], type = 'l', main = 'v')
matplot(result[[2]], type = 'l', main = 'u')

```



```
par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(noisy_data, main = "Noisy image")
image(result[[2]]%*%result[[1]], main = "Reconstructed image")

persp(noisy_data, theta = 40, main = "Noisy persp")
persp(result[[2]]%*%result[[1]], theta = 40, main = "Reconstructed persp")
```



Based on the images shown above, level of sparsity for u needs to be 25. Now, we can implement sparsity on u :

```
sparse_tuning_u <- attr(object_list, "Sparsity_parameter")

S_alpha_v <- list()
for (i in 1:n_var) {
  S_alpha_v[[i]] <- diag(ncol[,i])
}

CV_score_sparse_u <- CV_score_sparse_v <- GCV_score_smooth_u <- GCV_score_smooth_v <- Inf
CV_scores_result_u <- CV_scores_result_v <- c()
result = c()

# Step 1: Sparsity on u with no smoothness
for (sparse_tuning_single in sparse_tuning_u) {
  sparse_score = cv_sparse_row(data=X_temp,
                              ncol = ncol,
                              n_var = n_var,
                              S_alpha_v = S_alpha_v,
                              S_alpha_u = diag(nrow(X_temp)),
                              K_fold = K_fold,
                              sparse_tuning_result_u = sparse_tuning_single,
                              sparse_tuning_result_v = rep(0, n_var),
                              sparse_tuning_type = sparse_tuning_type)

  CV_scores_result_u <- c(CV_scores_result_u, sparse_score)
```

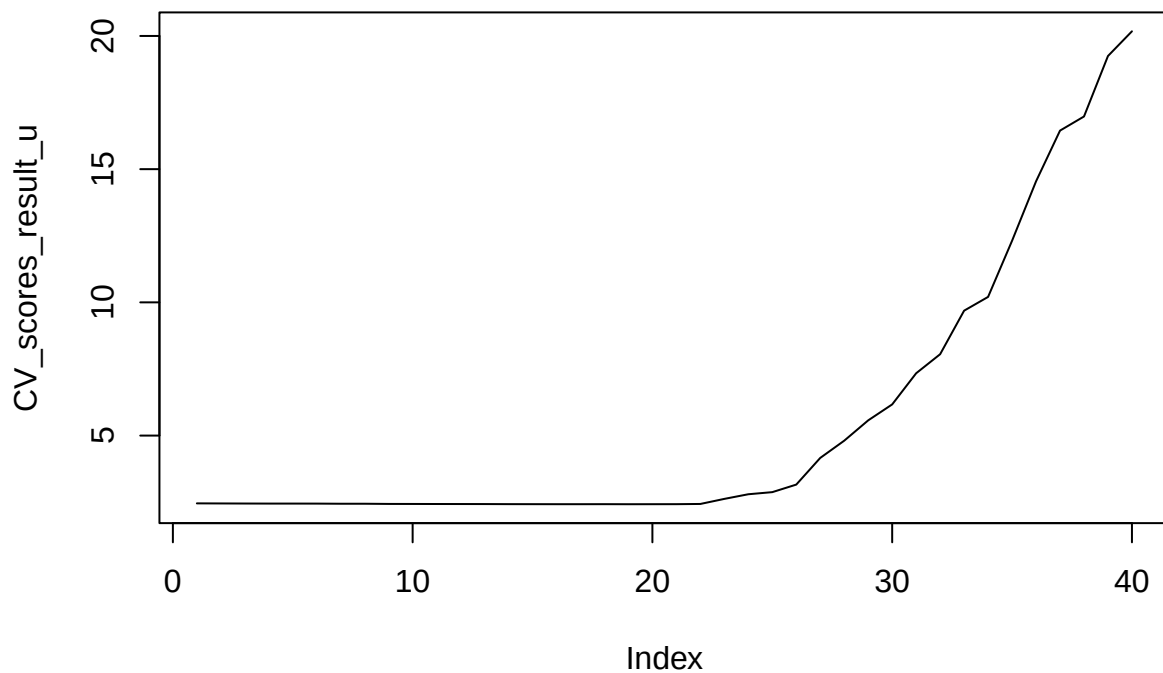
```

    if (sparse_score <= CV_score_sparse_u) {
      CV_score_sparse_u = sparse_score
      gamma_u = sparse_tuning_single
    }
  }
  gamma_u

```

```
## [1] 23
```

```
plot(CV_scores_result_u, type = 'l')
```



```

par(mfrow = c(3,2), mar = c(4, 4, 2, 1))
matplot(svd(z)$v[,1], type = 'l', main = "True V")
matplot(-svd(z)$u[,1], type = 'l', main = "True U")

matplot(svd(noisy_data)$v[,1], type = 'l', main = "True V")
matplot(svd(noisy_data)$u[,1], type = 'l', main = "True U")

result <- power_algo(data = X_temp,
  n_var = n_var,
  ncol = ncol,
  sparse_tuning_result_u = gamma_u, # A fixed number
  sparse_tuning_result_v = gamma_v, # A vector (length = p)
  S_alpha_v = S_alpha_v, # A list (length = p)
  S_alpha_u = diag(nrow(X_temp)), # A matrix
  sparse_tuning_type = sparse_tuning_type)

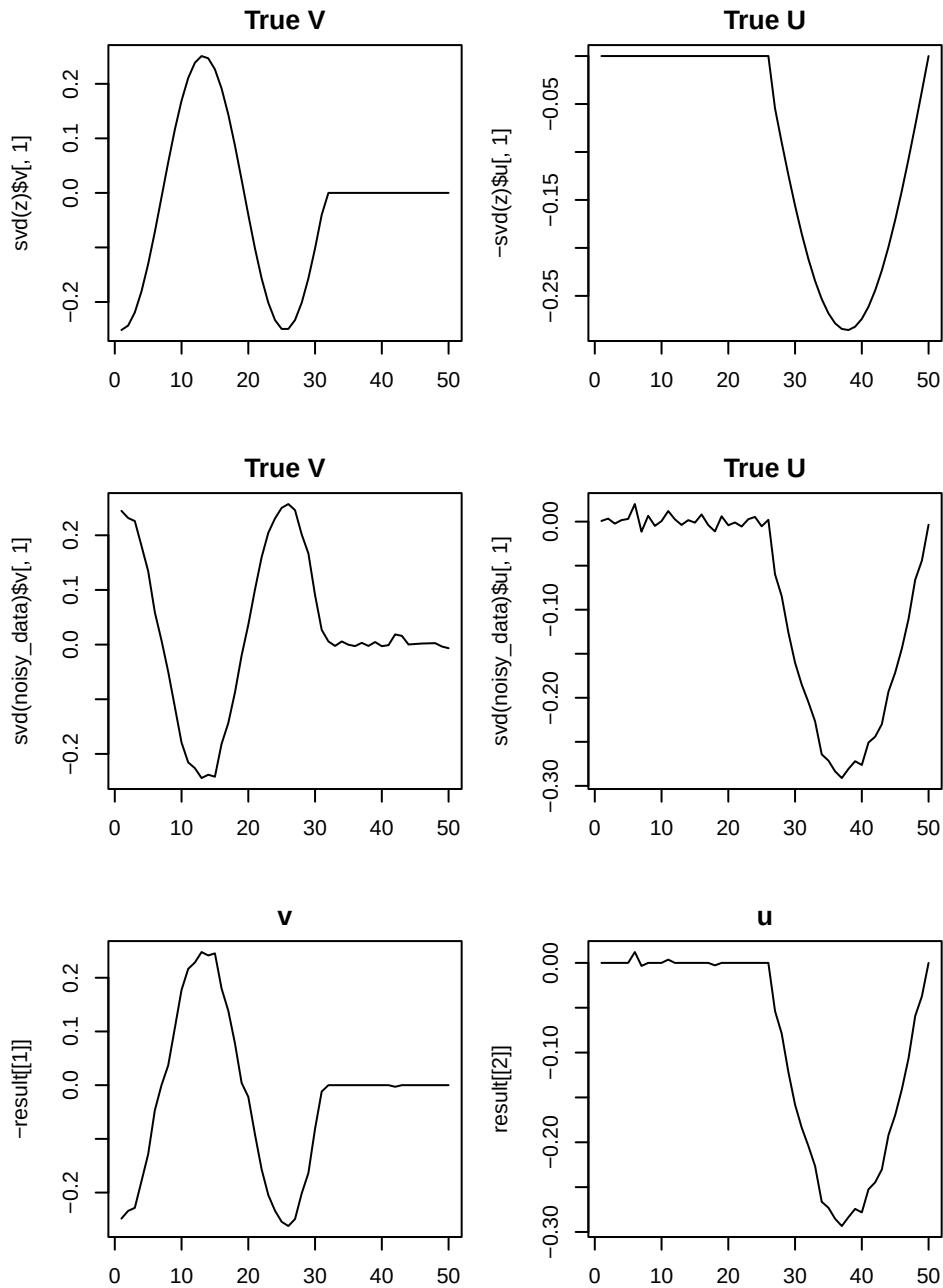
```



```

matplot(-result[[1]], type = 'l', main = 'v')
matplot(result[[2]], type = 'l', main = 'u')

```

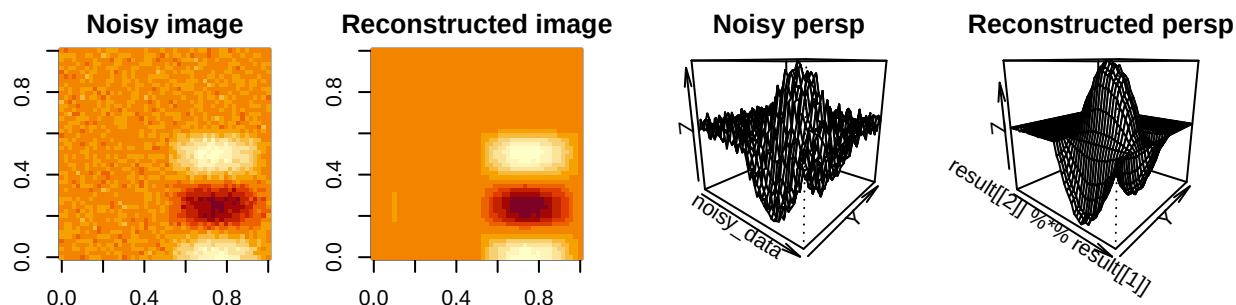


```

par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(noisy_data, main = "Noisy image")
image(result[[2]]%*%result[[1]], main = "Reconstructed image")

persp(noisy_data, theta = 40, main = "Noisy persp")
persp(result[[2]]%*%result[[1]], theta = 40, main = "Reconstructed persp")

```



Now, our goal is to add 1 SE to the CV formula:

```
cv_sparse_row_SE <- function(data,
                             n_var,
                             ncol,
                             S_alpha_u,
                             K_fold,
                             sparse_tuning_result_u, # vector of candidate sparsity levels for rows (u)
                             sparse_tuning_result_v, # fixed right sparsity (vector)
                             sparse_tuning_type) {

  set.seed(123)
  shuffled_cols <- sample(ncol(data)) # Shuffle column indices
  group_size <- ceiling(length(shuffled_cols) / K_fold)
  m <- ncol(data)

  CV_errors <- numeric(length(sparse_tuning_result_u))
  SE_errors <- numeric(length(sparse_tuning_result_u))
  fold_errors_list <- vector("list", length(sparse_tuning_result_u))

  for (j in 1:length(sparse_tuning_result_u)) {
    gamma_u <- sparsity_row_list[j]
    fold_errors <- numeric(K_fold)

    for (k in 1:K_fold) {
      # Define indices for test columns
      cols_to_remove <- shuffled_cols[((k - 1) * group_size + 1):min(k * group_size, length(shuffled_cols))]
```

```

# Create training and test sets
data_train <- data[, -cols_to_remove]
data_test  <- data[, cols_to_remove]

# Update column structure
updated_ncol <- update_ncol(ncol, cols_to_remove)

# Create identity smoother matrices (no smoothing)
S_alpha_v_train <- lapply(1:n_var, function(i) diag(updated_ncol[, i]))

# Run power algorithm
power_result <- power_algo(data = data_train,
                           n_var = n_var,
                           ncol = updated_ncol,
                           sparse_tuning_result_u = gamma_u,
                           sparse_tuning_result_v = sparse_tuning_result_v,
                           S_alpha_v = S_alpha_v_train,
                           S_alpha_u = S_alpha_u,
                           sparse_tuning_type = sparse_tuning_type)

u_hat <- power_result[[2]]
v_test <- t(as.matrix(data_test)) %*% u_hat

# Compute reconstruction error for this fold
fold_errors[k] <- sum((data_test - u_hat %*% t(v_test))^2) / ncol(data_test)
}

# Compute CV error and standard error for current gamma
CV_errors[j] <- mean(fold_errors)
SE_errors[j] <- sd(fold_errors) / sqrt(K_fold)
fold_errors_list[[j]] <- fold_errors
}

# One-standard-error rule selection
j_star <- which.min(CV_errors)
CV_1se_threshold <- CV_errors[j_star] + SE_errors[j_star]
j_1se <- max(which(CV_errors <= CV_1se_threshold)) # most regularized (largest gamma)
gamma_u <- sparse_tuning_result_u[j_1se]

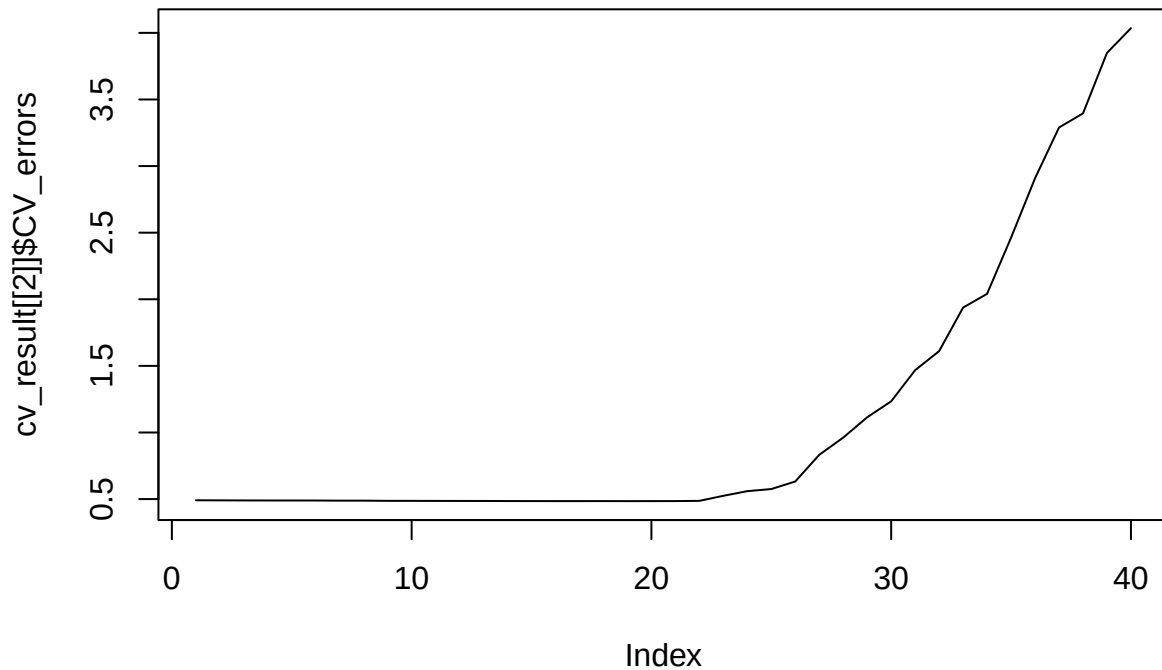
return(list(gamma_u, data.frame(sparse_tuning_result_u, CV_errors, SE_errors)))
}

cv_result <- cv_sparse_row_SE(data = X_temp,
                             n_var = n_var,
                             ncol = ncol,
                             S_alpha_u = diag(nrow(X_temp)),
                             K_fold = K_fold,
                             sparse_tuning_result_u = sparsity_row_list,
                             sparse_tuning_result_v = rep(0, n_var),
                             sparse_tuning_type = sparse_tuning_type)

```

```
gamma_u <- cv_result[[1]]
gamma_u

## [1] 26
plot(cv_result[[2]]$CV_errors, type = 'l')
```

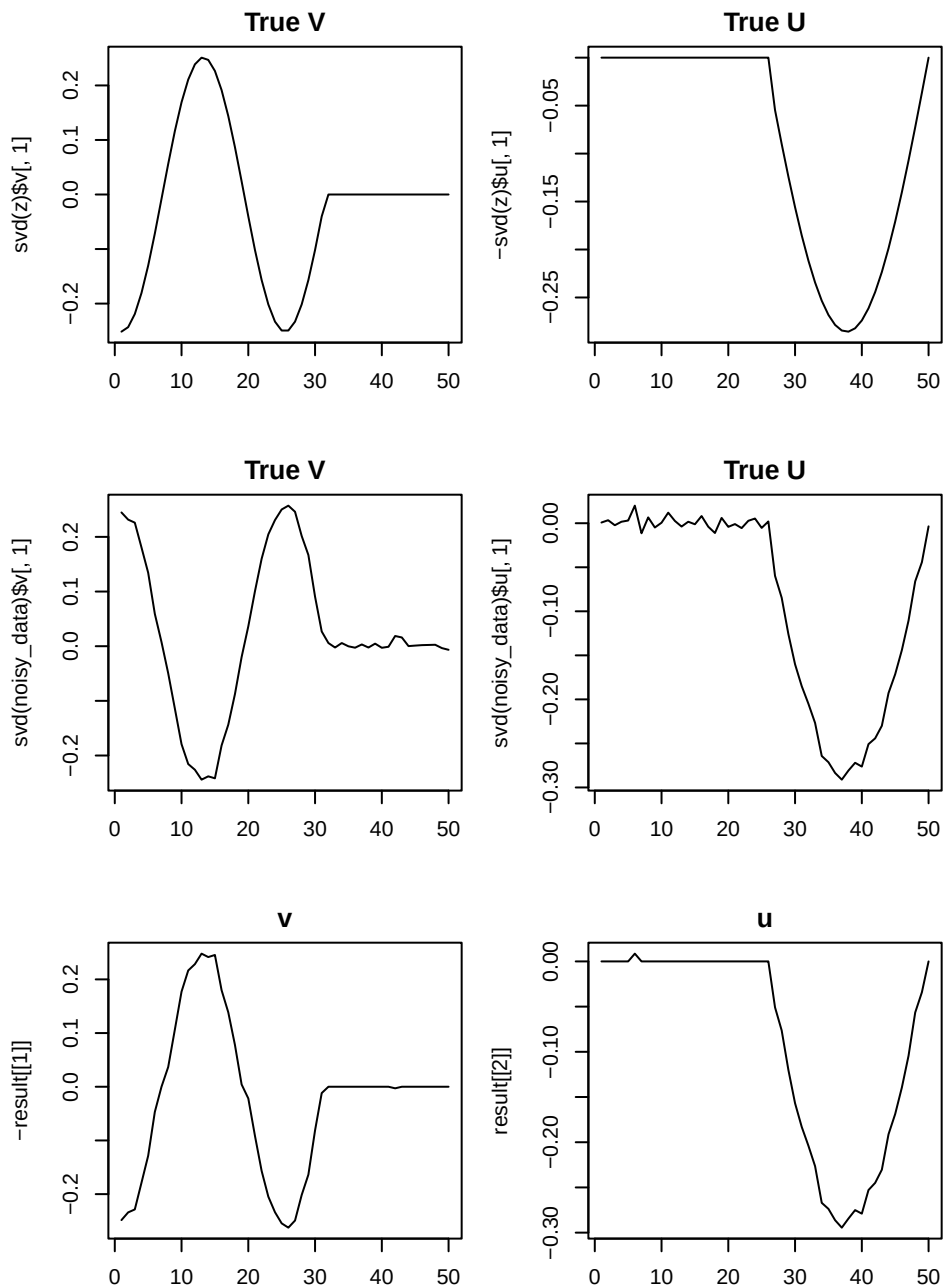


```
par(mfrow = c(3,2), mar = c(4, 4, 2, 1))
matplot(svd(z)$v[,1], type = 'l', main = "True V")
matplot(-svd(z)$u[,1], type = 'l', main = "True U")

matplot(svd(noisy_data)$v[,1], type = 'l', main = "True V")
matplot(svd(noisy_data)$u[,1], type = 'l', main = "True U")

result <- power_algo(data = X_temp,
                     n_var = n_var,
                     ncol = ncol,
                     sparse_tuning_result_u = gamma_u, # A fixed number
                     sparse_tuning_result_v = gamma_v, # A vector (length = p)
                     S_alpha_v = S_alpha_v, # A list (length = p)
                     S_alpha_u = diag(nrow(X_temp)), # A matrix
                     sparse_tuning_type = sparse_tuning_type)

matplot(-result[[1]], type = 'l', main = 'v')
matplot(result[[2]], type = 'l', main = 'u')
```



```
par(mfrow = c(1,4), pin = c(1,1), mar = c(2, 2, 2, 2), pty = "s")
image(noisy_data, main = "Noisy image")
image(result[[2]]%*%result[[1]], main = "Reconstructed image")

persp(noisy_data, theta = 40, main = "Noisy persp")
persp(result[[2]]%*%result[[1]], theta = 40, main = "Reconstructed persp")
```

